AI ACADEMY · PYTHON READINESS BRIDGE · STUDENT LESSON

Return values and local state

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Python readiness portfolio

THE CONNECTED WEEK

Return the data. Store it in the caller. Trace the scope.

Use returned values without confusing display, missing returns and local names.

Keep one habit: Two-cupboard return → extra-call trace → six return/scope cases → selected missing-return test → portfolio record.

MAIN LESSON · SECTION 1

Which value returns, and which name belongs only to the call?

> Keep displayed output, returned data and caller bindings separate.

Your learning goals

I can use a returned value in a later calculation.

I can trace **local variable**s separately for each function call.

I can detect a missing return and test repeated calls.

Recall before notes

- Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.
- Without opening the old lesson, explain one key idea from Functions and arguments. Give an example and a mistake that the idea helps you avoid.
- Without opening the old lesson, explain one key idea from Loops and trace tables. Give an example and a mistake that the idea helps you avoid.
- Without opening the old lesson, explain one key idea from Expressions and operator order. Give an example and a mistake that the idea helps you avoid.
- After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.
- This week you will practise:
- Use a returned value in a later calculation
- Trace **local variable**s separately for each function call
- Detect a missing return and test repeated calls
- A function show_remaining(capacity, used) prints capacity - used. What does calling it with (9, 4) display? Does defining it alone display that result?

My starting explanation

Your end goal

Trace **return value**s and local state and explain the changed program before running it.

▼ Earlier wording, alternatives and picture transcripts

Your learning target

Remember before reading

Retrieve what you already know

This week's end goal

MAIN LESSON · SECTION 2

Add spare spaces using returned values

> The caller can store a result without copying a number from the screen.

Read the two-cupboard task

Two fictional cupboards have spare spaces. We want to add their spare spaces without copying numbers from the screen. Last week a function printed each answer.

This week a function returns its answer so the calling code can store it, combine it or choose to print it.

State assumptions and saved-script behavior

Assume nonnegative integer counts and used no greater than capacity. This calculation does not validate those assumptions.

Work in a Python script so a call does not automatically display its **return value** as it might in an interactive console.

Your first explanation

List the given facts and one fact that is not supplied.

MAIN LESSON · SECTION 3

Return hands back data; the caller chooses how to use it

> A caller assignment stores the returned value under a different name.

Complete return-and-combine program

```
def remaining(capacity, used):
    spaces = capacity - used
    return spaces

first = remaining(12, 4)
second = remaining(10, 7)
print(first + second)
```

Trace the two local calculations

Two calls return two values

CALL	INSIDE THIS CALL	BACK IN THE CALLER
remaining(12, 4)	spaces becomes 8	first receives 8
remaining(10, 7)	spaces becomes 3	second receives 3

The caller adds the returned values. Only print(first + second) displays 11.

Separate return, assignment and display

Execution event	Returned value	Caller binding / displayed output
remaining(12,4) computes local spaces	8	Assignment binds first=8; no output
remaining(10,7) computes its own spaces	3	Assignment binds second=3; first still 8
print(first+second)	No new function result needed	Display 11
Remove outer print only	Return values still stored	No output from this script

Read execution after return

return spaces ends the current function call and sends the value of spaces back. The assignment first = remaining(12, 4) waits for that value and stores 8. The next call stores 3 in second. Neither call prints.

The final statement prints 11. A statement after an executed return in the same call is not reached.

Explain the deciding step

- Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

▼ **Earlier wording, alternatives and picture transcripts**

Connect the picture and the explanation

MAIN LESSON · SECTION 4

Compare returned data with a function that only prints

> A visible number does not establish what the call returned.

Predict the supported change

- Using the correct definition, store a = remaining(8, 3), b = remaining(8, 8), c = remaining(8, 3). State a, b and c. In a fresh script, explain why print(spaces) outside the function fails.

Keep the procedure fixed. Explain what changed in the inputs and result.

Predict the pictured variant

- Change: Add print(result) after the calls in a fresh script with no global result.

▼ Check after predicting

Prediction: NameError occurs because that local name is not defined in the **caller**.

Reason: A returned value and the function’s internal name are different things.

▼ Earlier wording, alternatives and picture transcripts

Compare a familiar case with a changed case

Change one thing in the pictured case

MAIN LESSON · SECTION 6

Trace a local result through two independent calls

> An outside print cannot read the function’s local name in this fresh script.

Read the complete extra-call pair and scope

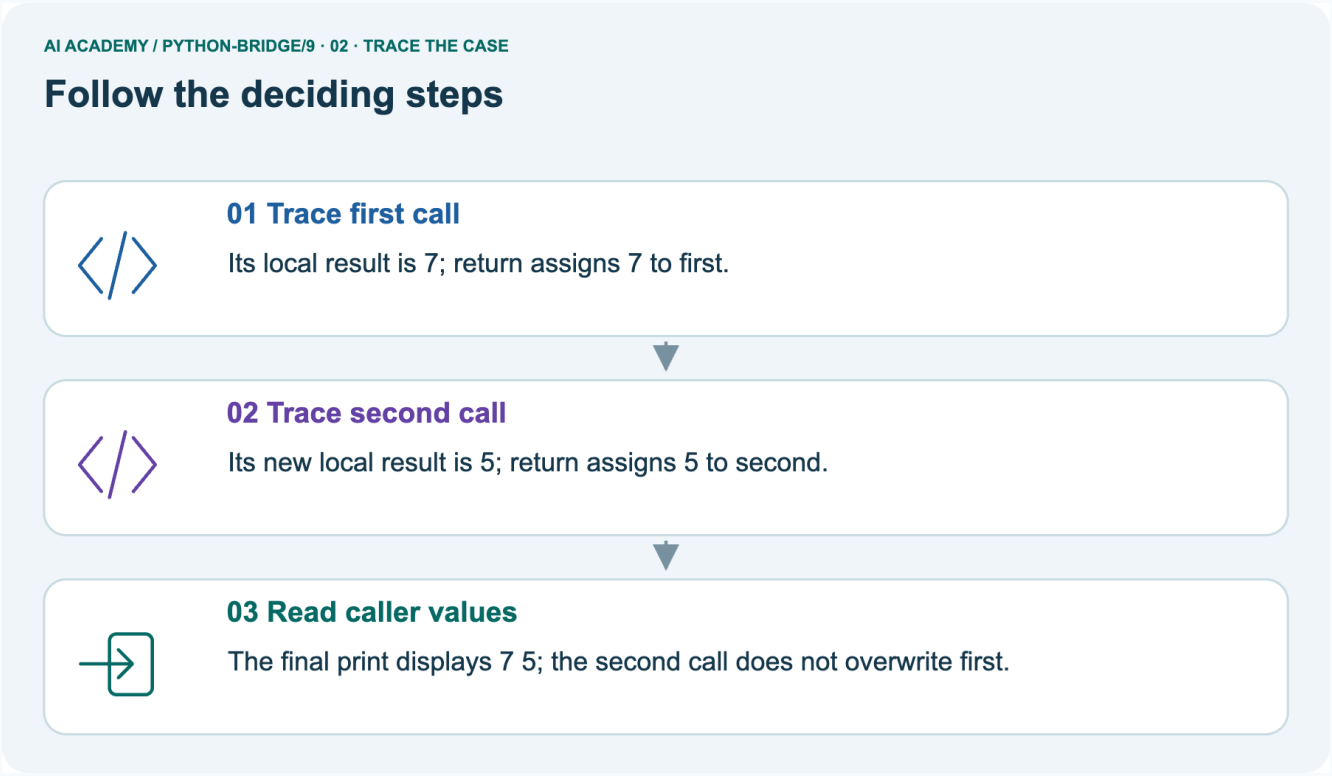
Return values survive their calls. A function returns a local calculation that the **caller** stores.

Python 3: def extra(a,b): result=a+b; return result. Then first=extra(2,5); second=extra(4,1); print(first,second).

Keep the scripts separate. Main remaining returns 8 and 3 to first/second, then prints 11. The pictured extra returns 7 and 5, then prints 7 5. W2 stores remaining results 5,0,5 in a/b/c. Each script starts from its own complete definition and calls. In the picture return passes 7 back; the caller’s first=extra(...) assignment binds first. A later local result does not overwrite that stored integer.

This lesson uses immutable numbers; shared mutable objects need separate reference reasoning.

Trace the labelled local/caller steps



Follow the labelled picture with the matching explanation. Open the vector diagram

Read the picture one step at a time

STEP	WHAT HAPPENS AND WHY
1. Trace first call	Its local result is 7; return assigns 7 to first.
2. Trace second call	Its new local result is 5; return assigns 5 to second.
3. Read caller values	The final print displays 7 5; the second call does not overwrite first.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Returning passes a value back to the **caller**. Local names belong to a function call; storing the returned value creates a separate **caller** binding.

Complete two-call program

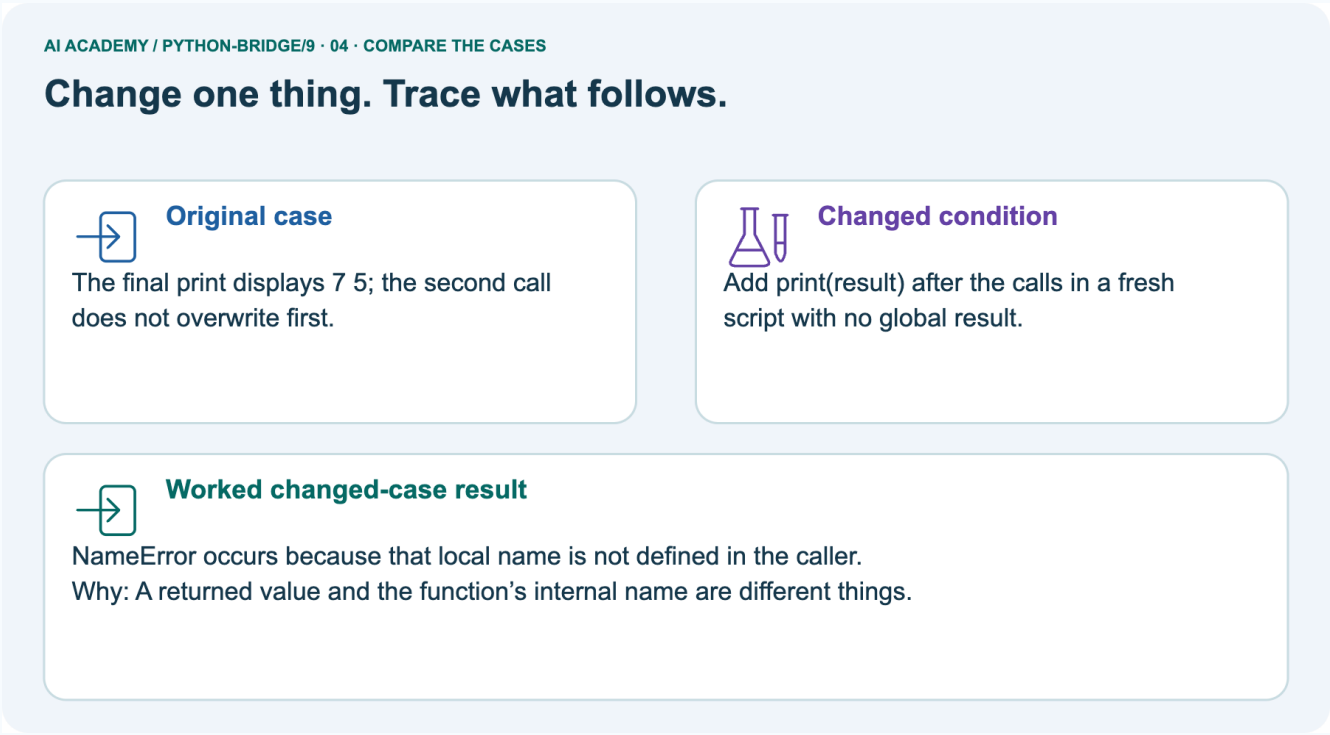
```
def extra(a, b):
    result = a + b
    return result

first = extra(2, 5)
second = extra(4, 1)
print(first, second)
```

Predict outside access to result

- Change one thing: Add print(result) after the calls in a fresh script with no global result. Predict the result before checking the explanation. What remains the same?

▼ Check after predicting



Follow the labelled picture with the matching explanation. Open the vector diagram

State immutable-value limits

- The example uses immutable integers; shared mutable objects require additional reasoning about references.

- Read the labelled picture: Trace the two returned values and explain why first stays seven. What happens if the **caller** tries to print result directly?

▼ Reference · Main spaces/caller trace

Point in execution	Local spaces	Caller values
First call before return	8	first not assigned yet
Second call before return	3	first is still 8
After both calls	Not accessible by this local name	first is 8 and second is 3

capacity, used and spaces belong to the current call. The second call gets new parameter values and calculates its own spaces; it does not overwrite first.

In a fresh script with no outside variable called spaces, adding print(spaces) after the calls raises NameError. Use the returned value stored in first or second instead.

> **Change the calls to remaining(9, 9) and remaining(9, 0). What values are stored and what does their sum print? What happens if only the final print statement is removed?**

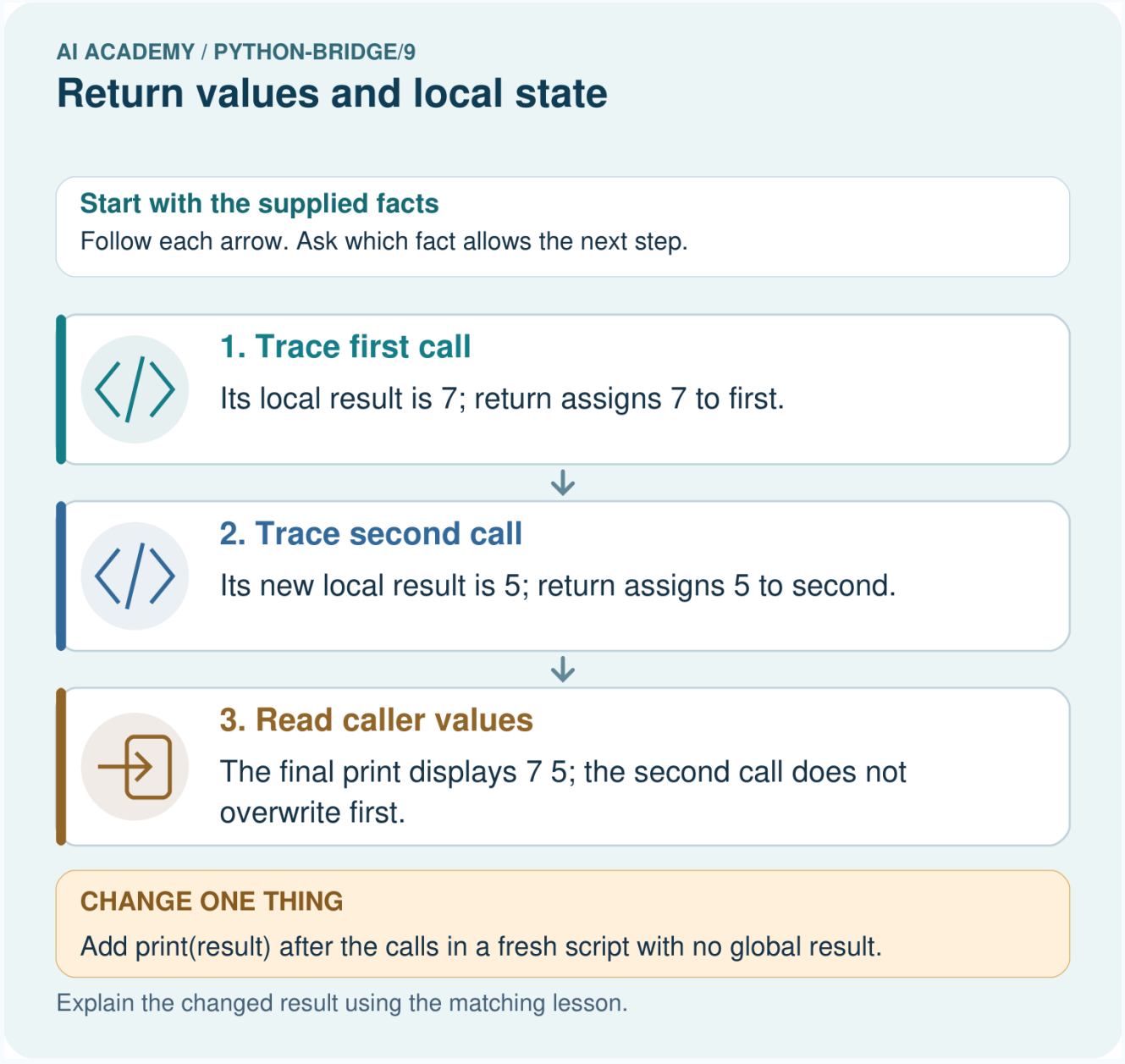
▼ Earlier wording, alternatives and picture transcripts

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Return values and local state

The facts we will use



Follow the labelled picture with the matching explanation. Open the vector diagram

Why this result follows

What this example does not establish

MAIN LESSON · SECTION 7

Find print used where the caller needs a return

> None is neither a measured zero nor the displayed number.

Read the complete faulty print-only function

```
def remaining(capacity, used):
    spaces = capacity - used
    print(spaces)

answer = remaining(12, 4)
print(answer)
```

Trace display 8 then stored None

This faulty version displays 8 and then **None**. It reaches the end of its body without a return statement, so the call returns **None**. Printing 8 inside the function does not make 8 its **return value**. **None** is not zero.

Trying answer + 1 raises TypeError in this example.

Repair return and keep the caller's print

Replace print(spaces) inside the function with return spaces. Keep print(answer) in the **caller**. The repaired script prints 8 once, and the **caller** can calculate answer + 1.

Test the stored value as well as what appears on screen.

Explain the failed addition and corrected result

- Use the faulty printing version with answer = remaining(11, 2). State the displayed number and the value stored in answer. Explain why print(answer + 1) fails, then repair the function and predict the repaired **output**.

State one repair and the checks you would rerun.

▼ **Earlier wording, alternatives and picture transcripts**

Locate the earliest unsupported step

MAIN LESSON · SECTION 8

Practise separate returned results with less help

> Keep the local name and caller variable labels explicit.

Explain the procedure

- Using the correct worked definition, execute result = remaining(15, 6), then doubled = result * 2. State both stored values and what the script displays before a print statement is added.

Change the calls to remaining(9, 9) and remaining(9, 0). What values are stored and what does their sum print? What happens if only the final print statement is removed?

Explain your prediction to a partner before comparing results.

Complete W2 separate-call trace

Use workbook W2 for the connected example “**Return value**s and local state”. First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan	Do	Check
What is given? Which rule or procedure applies?	Record each intermediate step.	Does the result follow? What remains unknown?

Audit printing-equals-return reasoning

- Use the worked case for **Return value**s and local state. Proposed explanation: “Printing a value returns that value”. Decide whether this explanation is supported.

  Point to the step or supplied fact that decides; explain your repair if needed.

- Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the pictured local-name boundary

- Trace the two returned values and explain why first stays seven. What happens if the **caller** tries to print result directly?

My explanation using a pictured step and the given facts

Reserve selected work

This is supported practice. Keep the separately named independent case for an attempt before feedback.

▼ **Earlier wording, alternatives and picture transcripts**

Practise with less help: Trace separate calls

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

Check your reasoning

Explain the picture

MAIN LESSON · SECTION 9

Test six complete return and local-scope cases

> Record any output before failure as well as statements never reached.

All supplied event and failure cases

Complete case	Expected caller/output behavior
A · Correct W2 calls remaining(8,3),(8,8),(8,3)	Store 5,0,5; final print shows 5 0 5

Complete case	Expected caller/output behavior
B · Add print(spaces) after correct W2 in a fresh script	First display 5 0 5, then NameError; local spaces not a caller name
C · Supplied faulty print-only remaining(12,4)	Body prints 8; caller receives None and prints None
D · Faulty call result answer, then print(answer+1)	TypeError during addition; no sum printed
E · Correct main script with final outer print removed	Calls compute and return values; no displayed output
F · Put print("after") immediately after an executed return inside a function	That body statement is not reached; outer calls may still display their results

Complete repeated-call W2 script

```
def remaining(capacity, used):
    spaces = capacity - used
    return spaces

a = remaining(8, 3)
b = remaining(8, 8)
c = remaining(8, 3)
print(a, b, c)
```

Predict, trace or run and repair the return boundary

1. Predict A and trace each local call before running the complete W2 script. For B, append only the outside local-name access and preserve the valid output before its error. For C/D, use the supplied faulty definition in a fresh file, separating what it displays from the None it returns. Repair one statement, then test both the stored value and the caller's arithmetic, including legitimate zero.

   E/F probe output placement and return's control flow. State whether each record is predicted, paper-traced or executed. The capacity example still assumes valid supplied counts; return does not validate inputs or train a model. Keep the selected twice-script task unused until the independent attempt.

Keep the evidence record

Using the correct definition, store a = remaining(8, 3), b = remaining(8, 8), c = remaining(8, 3). State a, b and c. In a fresh script, explain why print(spaces) outside the function fails.

Record	Your evidence
Given input and fixed rule	
Prediction before checking	
Actual result, or “not run”	
First mismatch and one explanation	
Boundary or missing input and expected response	

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

▼ Reference · Original correct and faulty programs

```
def remaining(capacity, used):
    spaces = capacity - used
    return spaces

first = remaining(12, 4)
second = remaining(10, 7)
print(first + second)
```

```
def remaining(capacity, used):
    spaces = capacity - used
    print(spaces)
```

```
answer = remaining(12, 4)
print(answer)
```

▼ Earlier wording, alternatives and picture transcripts

Predict, investigate and preserve the evidence

Plan → try → check

MAIN LESSON · SECTION 14

Try alone: trace selected returns and missing-return behavior

> Separate successful output, argument-evaluation failure and local-scope failure.

Selected complete task and scope

Trace both calls and the final expression. If return total is removed from the function, does this final print complete? Would a separate print(total) outside the function have access to its **local variable**?

```
def twice(n):
    total = 2 * n
    return total
first = twice(3)
second = twice(5)
print(first, second, first + second)
```

Trace this selected definition and both caller assignments from a fresh script. Removing return is a separate changed program: evaluate all arguments to the final print before assuming any line is displayed. A separate outside local-name access is another distinct test, not part of the original successful run.

Your response and help record

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Use feedback afterwards

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

▼ Additional practice · Earlier return transfer

Each tray holds 5 counters. Define stock(trays, loose) to return the total using a **local variable** total. Assume nonnegative integer arguments. Store results for (2, 3), (0, 3) and (2, 0).

Print the sum of the first two stored results. Predict that **output** and all three stored values. Call (2, 3) again to check fresh state.

My independent procedure, result and limitation

Record support used: **none**, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

▼ Earlier wording, alternatives and picture transcripts

Independent case: Use returns and keep local state local

Attempt this without hints

Use feedback to improve

MAIN LESSON · SECTION 11

Keep local calculations and caller results on the portfolio trace

> A displayed value and stored return may disagree in a faulty program.

This week's contribution

Add a labelled **return value**s and local state **evidence** sheet to Python readiness portfolio. Show how you can use a returned value in a later calculation.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

Project checkpoint and evidence

Project evidence	Record
Component and version	
Inputs permitted at prediction time	
Trace and comparison with earlier work	
Failure, limit and human review	

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

▼ **Earlier wording, alternatives and picture transcripts**

Evidence for your project

MAIN LESSON · SECTION 16

Finish: say what returned, what was stored and what was displayed

> Local computation and caller binding are different parts of the trace.

Explain without notes

- Close your notes. Explain this goal in your own words: Use a returned value in a later calculation

> **My deciding step, evidence and one remaining question**

Plan a changed-case return

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Reference · The words used in this procedure**

REFERENCE · SECTION 5

Reference · The words used in this procedure

> Explain each term using a particular code line.

Vocabulary and response

Word	Meaning
Return value	A value a function sends back to its caller
Caller	The code that calls a function
Local variable	A name assigned inside a function and used in that function call
Scope	The region of code in which a name can be looked up
None	A Python value representing no result value in this lesson

Use two terms to explain a step in this week's example.

▼ Extra challenge · Explain, apply and challenge

REFERENCE · SECTION 10

Extra challenge · Explain, apply and challenge

> Choose the next useful task after supported practice.

Choose a challenge

- Using the correct worked definition, execute result = remaining(15, 6), then doubled = result * 2. State both stored values and what the script displays before a print statement is added.

- Use the faulty printing version with answer = remaining(11, 2). State the displayed number and the value stored in answer. Explain why print(answer + 1) fails, then repair the function and predict the repaired **output**.

- A function body has spaces = capacity - used, then return spaces, then print("done"), all at the same indentation. What does answer = remaining(4, 4) store and display? Explain whether the final print can run.

> **Explain which evidence supports your conclusion and which claim remains unproved.**

▼ Earlier wording, alternatives and picture transcripts

Explain → apply → challenge

▼ Reference · Inspect your evidence

REFERENCE · SECTION 12

Reference · Inspect your evidence

> Use the record after practice rather than revealing the independent answer.

Self-check and explanation

- For a function that only calculates from these integer arguments, repeat the first call after a different call. remaining(12, 4), remaining(10, 7), remaining(12, 4) should return 8, 3, 8.

  Store each result and compare it with a **prediction**. These examples create fresh **local variable**s each time; do not assume every possible function is independent of earlier calls.

- A function body has spaces = capacity - used, then return spaces, then print("done"), all at the same indentation. What does answer = remaining(4, 4) store and display? Explain whether the final print can run.

> **A learner says that a function is correct because it prints 8. Explain how you would check that its caller actually receives 8. Include a second call and say why two passing tests are limited evidence.**

My limited claim and a useful next check

▼ **Extra practice · A complete trace**

REFERENCE · SECTION 13

Extra practice · A complete trace

> Predict before opening the supplied trace.

Explain the input and procedure

- Follow the **input** through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

My complete trace of the worked case

▼ Check the complete trace

Two calls return two values

CALL	INSIDE THIS CALL	BACK IN THE CALLER
remaining(12, 4)	spaces becomes 8	first receives 8
remaining(10, 7)	spaces becomes 3	second receives 3

The caller adds the returned values. Only print(first + second) displays 11.

▼ Earlier wording, alternatives and picture transcripts

Complete evidence record

▼ Reference · Earlier route checklist

REFERENCE · SECTION 15

Reference · Earlier route checklist

> Keep one main lesson route and use this as a reminder.

Checklist and first prediction

By the end, I can	How I will show it
Use a returned value in a later calculation	Explain the deciding step, show a trace or test, and state one limit.
Trace local variables separately for each function call	Explain the deciding step, show a trace or test, and state one limit.
Detect a missing return and test repeated calls	Explain the deciding step, show a trace or test, and state one limit.

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

▼ Earlier wording, alternatives and picture transcripts

Start with this question: **Return value**s survive their calls. A function returns a local calculation that the **caller** stores.

▼ Extra practice · Predict, check and repair

REFERENCE · SECTION 17

Extra practice · Predict, check and repair

> Use only after an attempt; keep the reserved assessment separate.

First question and hint

- **Return value**s survive their calls. A function returns a local calculation that the **caller** stores.

- Which labelled step matches this detail? The final print displays 7 5; the second call does not overwrite first.
- 1. Read **caller** values
- 2. Trace first call
- 3. Trace second call
- Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

Practice 1: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**

▼ **Check the first explanation**

After your attempt, compare with the supported explanation: Read **caller** values The final print displays 7 5; the second call does not overwrite first. Returning passes a value back to the **caller**.

Local names belong to a function call; storing the returned value creates a separate **caller** binding.

Changed-case question and hint

- Changed-case practice: Add print(result) after the calls in a fresh script with no global result. Which conclusion is supported by the supplied facts?
- 1. We can decide the result without examining the changed **input** or the procedure.

- 2. NameError occurs because that local name is not defined in the **caller**.
- 3. This one example guarantees correct results for every future **input**.
- Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

Practice 2: explain the deciding evidence

> **My selected conclusion, deciding fact and explanation**
>
> | |
> |---|
> | |
> | |
> | |
> | |

▼ **Check the changed explanation**

After your attempt, compare with the supported explanation: NameError occurs because that local name is not defined in the **caller**. A returned value and the function's internal name are different things.

Repair and keep the record

Review Functions and arguments (python-bridge/8). Goal: Distinguish a function definition from a function call. Short practice: Match every argument to a parameter for the two calls.

Why do **output**s differ, and what prints when both calls are removed?. Return to this week and explain what you changed.

Review Loops and trace tables (python-bridge/6). Goal: Trace variable values before and after each loop iteration. Short practice: List the four visited values and their running totals.

Explain why 5 is absent and what prints for range(1,1).. Return to this week and explain what you changed.

Review Expressions and operator order (python-bridge/3). Goal: Evaluate numeric expressions using Python's grouping rules. Short practice: Trace the operation order in both expressions.

Identify which picture step changes with parentheses and predict both results when the multiplier becomes four.. Return to this week and explain what you changed.

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

▼ **Earlier wording, alternatives and picture transcripts**

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

Repair a missing building block

Predict, run and debug

Keep your completed work

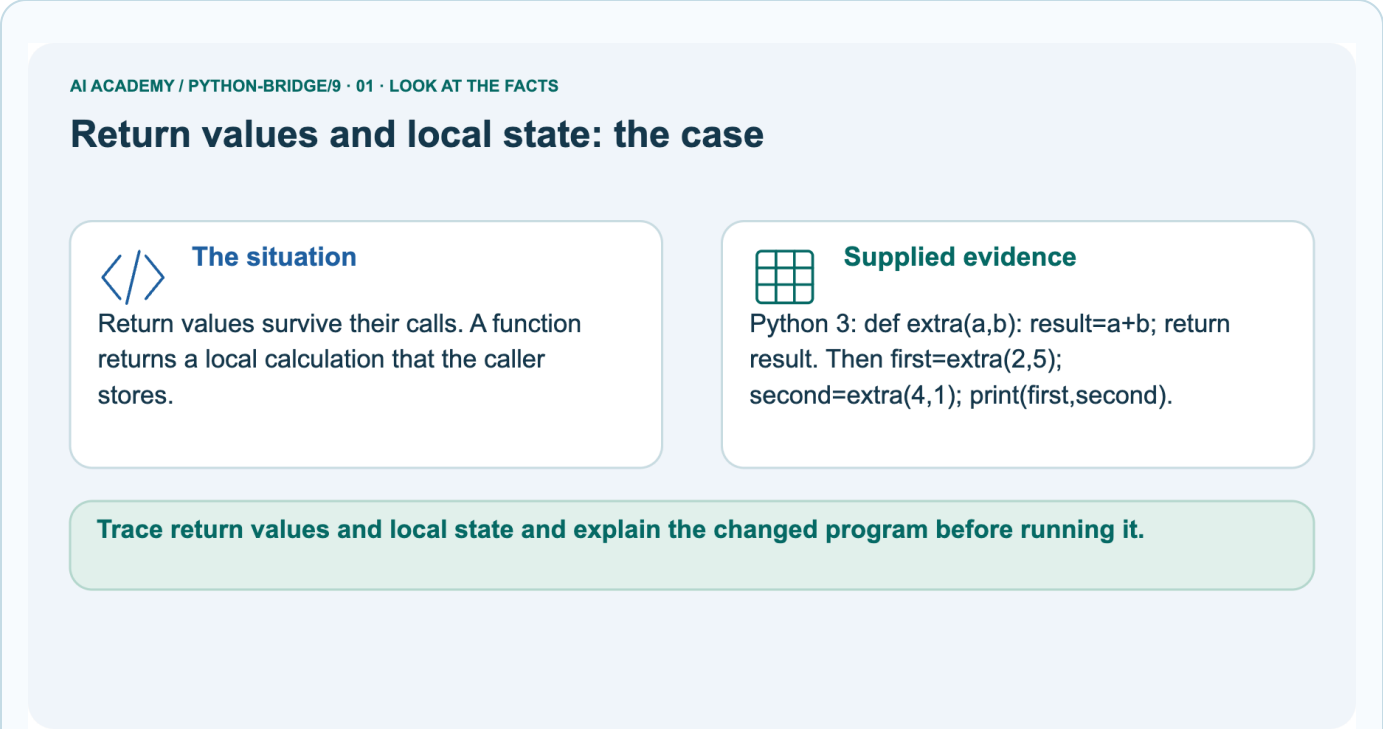
▼ **Reference · The six-picture journey**

REFERENCE · SECTION 18

Reference · The six-picture journey

> Choose the picture needed for the present question.

01. Return values and local state: the case



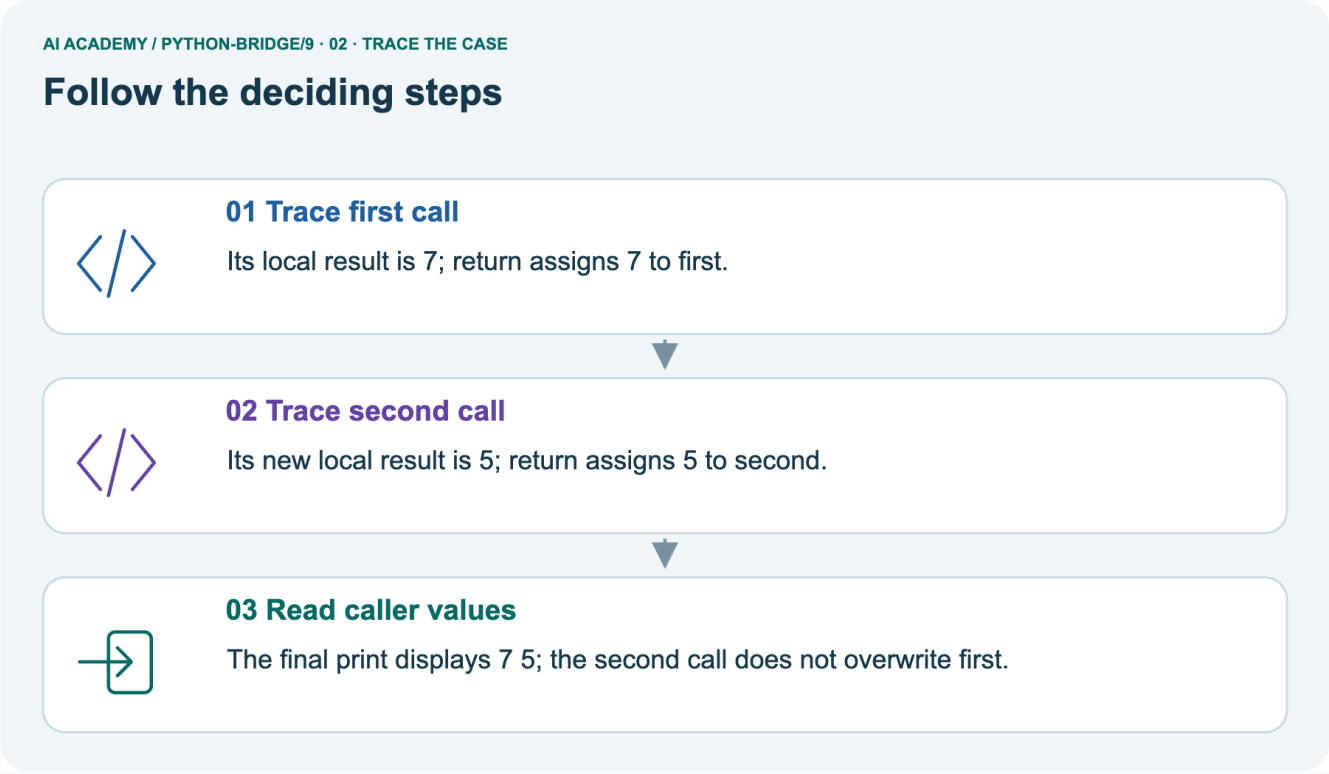
Follow the labelled picture with the matching explanation. Open the vector diagram

Return values survive their calls. A function returns a local calculation that the **caller** stores.
Supplied facts: Python 3: def extra(a,b): result=a+b; return result.

Then first=extra(2,5); second=extra(4,1); print(first,second).

Point to the given fact that matters most. Explain why it matters.

02. Follow the deciding steps

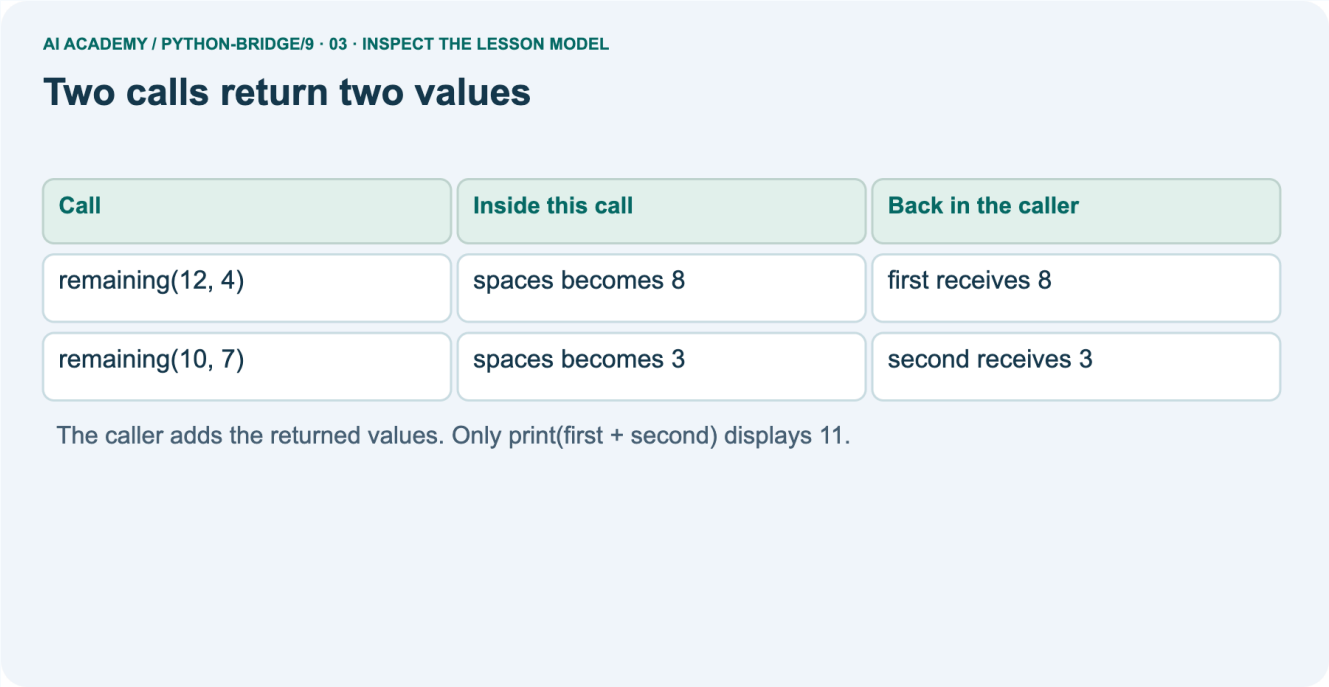


Follow the labelled picture with the matching explanation. Open the vector diagram

Trace first call: Its local result is 7; return assigns 7 to first. → Trace second call: Its new local result is 5; return assigns 5 to second. → Read **caller** values: The final print displays 7 5; the second call does not overwrite first.

Point to each numbered stage and explain which recorded **input** it uses.

03. Two calls return two values

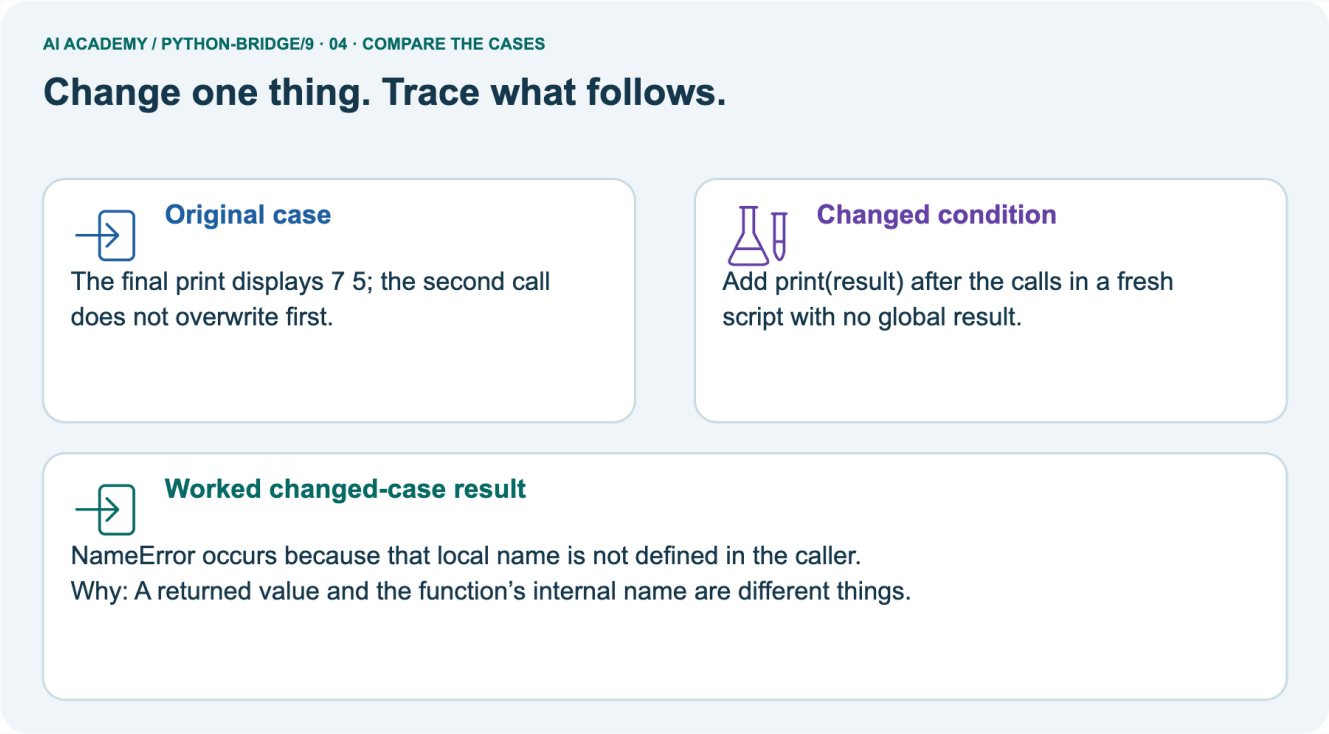


Follow the labelled picture with the matching explanation. Open the vector diagram

Two calls return two values. remaining(12, 4) | spaces becomes 8 | first receives 8. remaining(10, 7) | spaces becomes 3 | second receives 3

Read the labels and values. Explain the relationship the model shows.

04. Change one thing. Trace what follows.



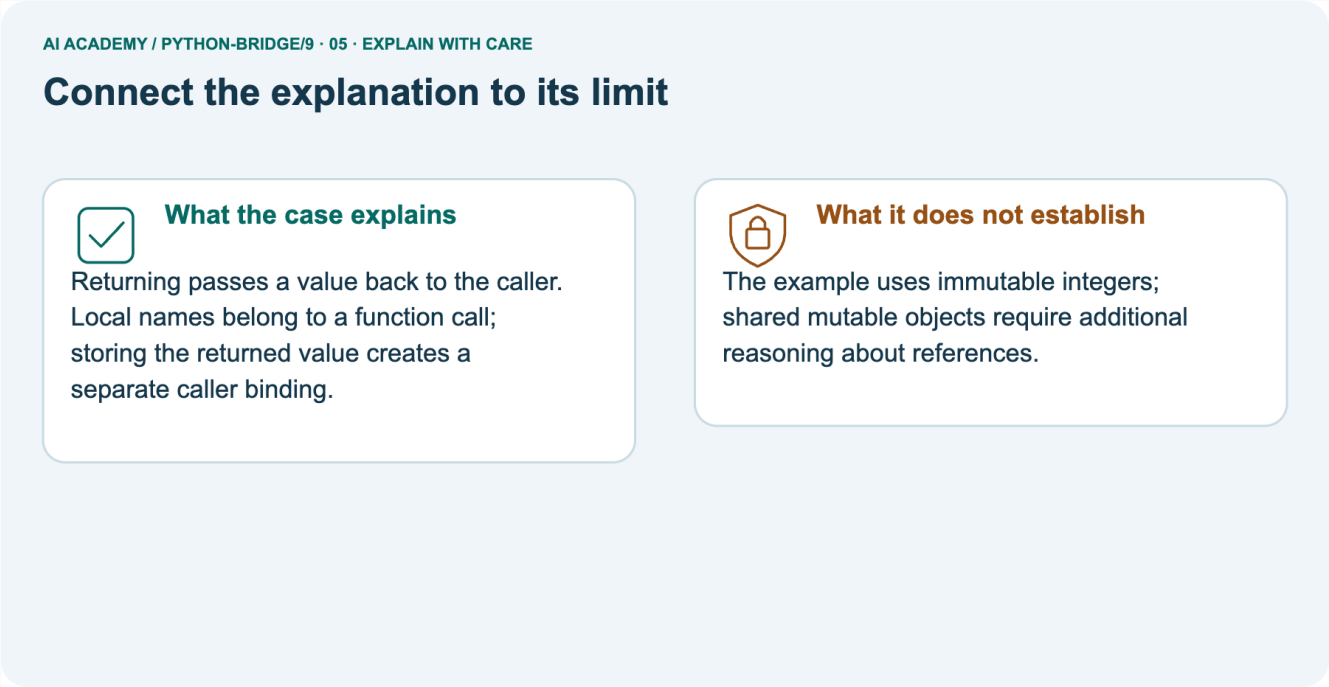
Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: The final print displays 7 5; the second call does not overwrite first. Changed condition: Add print(result) after the calls in a fresh script with no global result.

Expected result: NameError occurs because that local name is not defined in the **caller**.
Reason: A returned value and the function’s internal name are different things.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

05. Connect the explanation to its limit



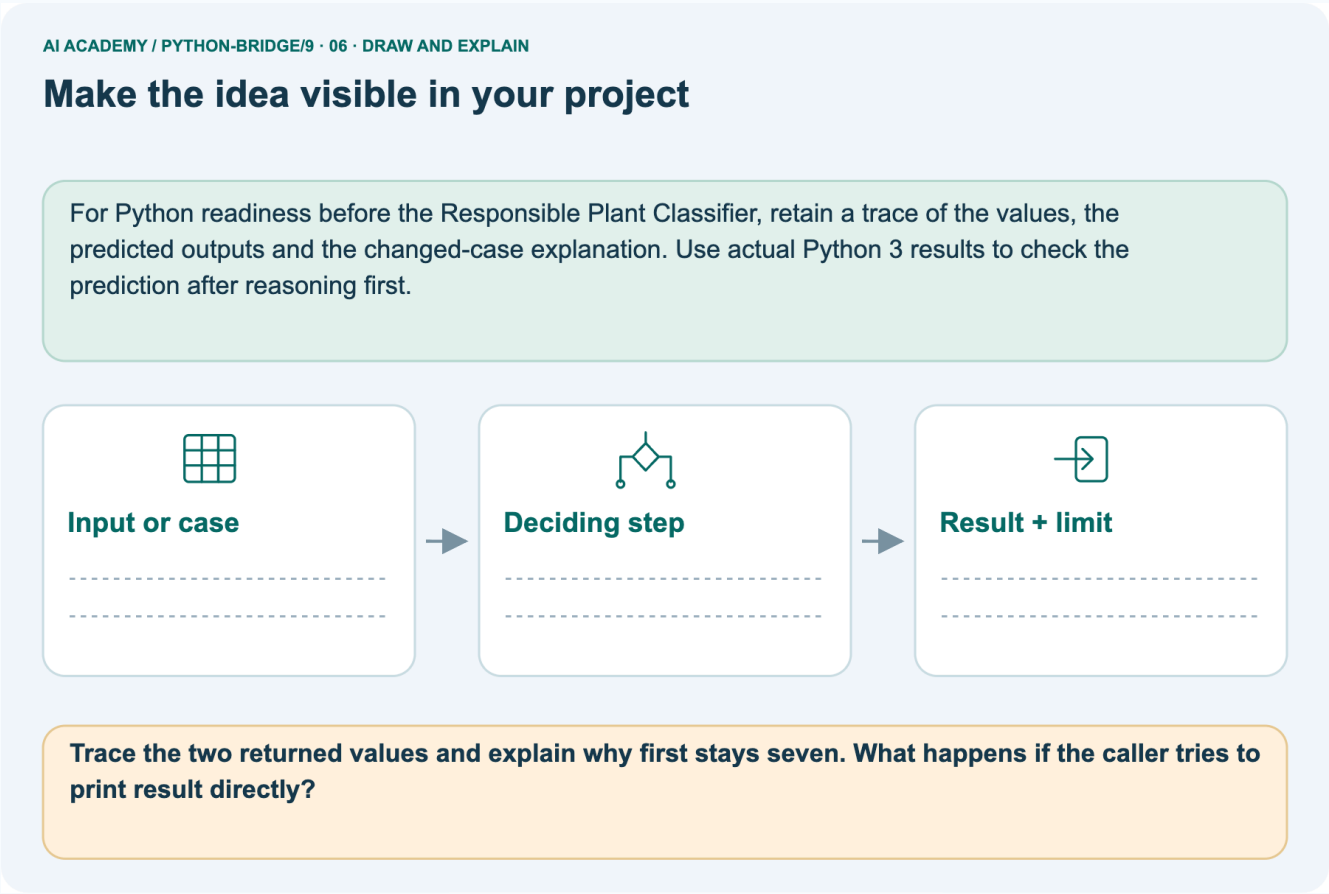
Follow the labelled picture with the matching explanation. Open the vector diagram

Returning passes a value back to the **caller**. Local names belong to a function call; storing the returned value creates a separate **caller** binding.

Limit: The example uses immutable integers; shared mutable objects require additional reasoning about references.

Explain the case, then point to a limit on the claim.

06. Make the idea visible in your project



Follow the labelled picture with the matching explanation. Open the vector diagram

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted **output**s and the changed-case explanation. Use actual Python 3 results to check the **prediction** after reasoning first.

Draw your **input**, deciding step, result and limit. Supported question: Trace the two returned values and explain why first stays seven. What happens if the **caller** tries to print result directly?

Draw your project trace and justify the deciding step using **evidence**.

▼ Earlier wording, alternatives and picture transcripts

01. Return values and local state: the case

02. Follow the deciding steps

03. Two calls return two values

04. Change one thing. Trace what follows.

05. Connect the explanation to its limit

06. Make the idea visible in your project

▼ Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.